

# SIT742 2026T2 Assignment 1 Specification: Critical Analysis

---

## Submission

Submit the required files in CloudDeakin via Assignments located under Assessment. The PDF must be exported from your completed notebook and include your code outputs, figures, tables, and written explanations.

## Assessment Support

For current Deakin information about assessment submission, extensions, special consideration, and late penalties, refer to the official Deakin assessment support page and the unit site.

## Contents

---

- Front matter
- Assignment overview
- Learning outcomes
- Assessment structure
- Dataset and resources
- Starter notebook and submission workflow
- Part I: Python Programming Tasks
- Part II: HK2012-2018 Dataset Analysis and Critical Interpretation
- Part III: Transfer and Reflection
- Back matter
- Submission requirements
- Video evidence
- Reproducibility requirements
- Academic integrity and GenAI policy
- Extensions, special consideration, and submission policy
- Submission checklist

## Front matter

---

### Assignment overview

| Item            | Details                    |
|-----------------|----------------------------|
| Assignment      | Assignment 1               |
| Title           | Critical analysis          |
| Unit            | SIT742 Modern Data Science |
| Teaching period | Trimester 2, 2026          |

|                     |                                                                                                                                                                                                |
|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Mode                | Individual                                                                                                                                                                                     |
| Weight              | 30% of the unit result                                                                                                                                                                         |
| Internal marks      | 100 marks                                                                                                                                                                                      |
| Due date            | Saturday 8 August 2026, 8pm AEST                                                                                                                                                               |
| Submission location | CloudDeakin > Assessment > Assignments                                                                                                                                                         |
| Required files      | Completed notebook (.ipynb) and exported PDF (.pdf). Students aiming for High Distinction may also submit an optional video; video may also be requested for verification. See Video evidence. |

In this assignment, you will complete six Python programming tasks, conduct a reproducible critical analysis of Hong Kong visitor-arrival and Search Intensity Index data, and reflect on how the functions, objects, and workflow could transfer to another TULIP Lab open-data dataset. The assignment is designed to assess your ability to write clear Python functions, load and validate public data, create reproducible analysis tables, compare markets, engineer interpretable features, and explain the limits and transferability of descriptive data-analysis workflows.

No formal modelling or forecasting is required in Assignment 1.

## Learning outcomes

This assignment assesses the following unit learning outcomes:

- ULO1: Critically explore and articulate advancements in new and emerging fields in modern data science.
- ULO2: Explain and critically analyse advanced concepts and theoretical frameworks in modern data science.
- ULO3: Evaluate modern data analytics techniques for their effectiveness in solving real-world problems.

## Assessment structure

| Part     | Component                                                | Marks |
|----------|----------------------------------------------------------|-------|
| Part I   | Python Programming Tasks                                 | 30    |
| Part II  | HK2012-2018 Dataset Analysis and Critical Interpretation | 60    |
| Part III | Transfer and Reflection                                  | 10    |
|          | Total                                                    | 100   |

Part I contains six questions worth 5 marks each. Part II contains a reproducible data-analysis task using the required market files. Part III is a discussion/reflection task only.

A student-facing public rubric will be provided on the unit site together with this specification and the starter notebook. It summarises the broad assessment expectations for each part and question.

## Dataset and resources

Use the public TULIP Lab HK2012-2018 dataset.

Dataset repository page:

<https://github.com/tulip-lab/open-data/tree/main/HK2012-2018>

Approved raw URL base for loading CSV files in Python:

```
https://raw.githubusercontent.com/tulip-lab/open-data/main/HK2012-2018
```

Use the repository page to inspect the dataset files and documentation. Use the raw URL base in your notebook when loading CSV files with Python. Do not use the GitHub repository page URL inside `pandas.read_csv()`.

Required market files:

| Market        | File              |
|---------------|-------------------|
| Australia     | Australia.csv     |
| United States | United_States.csv |
| Thailand      | Thailand.csv      |

Optional extension files:

| Market         | File               |
|----------------|--------------------|
| Philippines    | Philippine.csv     |
| Singapore      | Singapore.csv      |
| United Kingdom | United_Kingdom.csv |

Optional extension markets may be used to add further comparative evidence. They are not required for full marks or High Distinction if the required three-market analysis is already excellent.

Dataset source and acknowledgement:

```
TULIP Lab HK2012-2018 dataset. Dataset associated with:  
Zhang, Li, Muskat, and Law (2020), Tourism Demand Forecasting: A Decomposed Deep Learning Approach,  
Journal of Travel Research.  
DOI: 10.1177/0047287520919522
```

Your notebook must clearly acknowledge the dataset source.

## Starter notebook and submission workflow

You must complete the provided starter notebook for this assignment. The starter notebook contains the required question structure, function stubs, object placeholders, and markdown prompts that correspond to this specification.

Use this specification as the authoritative description of the assessment requirements. Use the starter notebook as the working file in which you complete your code, tables, figures, and written answers.

You may add code cells and markdown cells where needed, but you should preserve the question headings, required function names, required object names, and submission structure. Replace all None placeholders with completed objects. Leaving a required object as None does not satisfy the requirement.

Your exported PDF must be generated from the completed notebook and should show your code outputs, figures, tables, and written explanations.

## Part I: Python Programming Tasks

Complete the six required functions in your starter notebook. Each function must return the required value and must not rely only on printed output. Do not use interactive input.

The Part I illustrative examples use small toy inputs to clarify the required behaviour. They are not hidden tests. Your functions should work for other valid inputs that follow the same specification.

### Question 1: 'normalise\_column\_labels(labels)' [5 marks]

Your task:

Write a function that converts a sequence of raw column labels into cleaned, analysis-ready column labels. The function must remove leading/trailing whitespace, convert labels to lowercase, replace internal whitespace with underscores, and add deterministic suffixes when cleaned labels are duplicated.

Required function signature:

```
def normalise_column_labels(labels):  
    pass
```

Input:

- labels: a sequence of column-label values.

Output:

- Return a tuple (cleaned\_labels, label\_map).
- cleaned\_labels must be a list of cleaned column names.
- label\_map must be index-aware so duplicate original labels can still be represented. Use a list of records, such as dictionaries with index, original, and cleaned keys.

Output field meanings:

- cleaned\_labels: cleaned labels in the same order as the input.
- label\_map: an index-aware mapping from each original input position to its cleaned label.

Constraints:

- Strip leading and trailing whitespace.
- Convert labels to lowercase.
- Replace one or more internal whitespace characters with one underscore.
- If cleaned labels are duplicated, use suffixes \_2, \_3, and so on.
- Preserve the original label order.

Illustrative example:

```
labels = [" Date ", "Arrival", "Search Index", "Search Index", "Arrival "]
```

Expected output format:

```
cleaned_labels = ["date", "arrival", "search_index", "search_index_2", "arrival_2"]

label_map = [
    {"index": 0, "original": " Date ", "cleaned": "date"},
    {"index": 1, "original": "Arrival", "cleaned": "arrival"},
    {"index": 2, "original": "Search Index", "cleaned": "search_index"},
    {"index": 3, "original": "Search Index", "cleaned": "search_index_2"},
    {"index": 4, "original": "Arrival ", "cleaned": "arrival_2"},
]
```

Dataset connection:

This function is useful in Part II because real CSV files may contain spaces, inconsistent capitalisation, or repeated-looking column names.

## Question 2: 'validate\_monthly\_date\_range(date\_values, start="2012-01", end="2018-12")' [5 marks]

Your task:

Write a function that checks whether a sequence of date values can be parsed as monthly dates and covers every month from the required start month to the required end month.

Required function signature:

```
def validate_monthly_date_range(date_values, start="2012-01", end="2018-12"):
    pass
```

Input:

- date\_values: a sequence of date-like values.
- start: required start month as a string such as "2012-01".
- end: required end month as a string such as "2018-12".

Output:

- Return a dictionary with deterministic keys.
- The dictionary must include at least is\_complete, start, end, n\_periods, missing\_months, and extra\_months.

Output field meanings:

- is\_complete: True if every expected month from start to end is present and no required month is missing; otherwise False.
- start: the required start month used for validation, formatted as YYYY-MM.
- end: the required end month used for validation, formatted as YYYY-MM.
- n\_periods: the number of distinct valid monthly periods found in date\_values after parsing and normalising to months.
- missing\_months: expected months between start and end that are not present in date\_values, formatted as YYYY-MM strings.
- extra\_months: parsed months that are present in date\_values but fall outside the required start to end range, formatted as YYYY-MM strings.

Constraints:

- Parse values into monthly periods or month-start timestamps.
- Check whether all expected months from start to end are present.
- Report missing months as strings such as "2012-03".
- Report months outside the expected range in extra\_months.
- If the same month appears more than once, count it once in n\_periods after parsing, but still detect missing and extra months correctly.

Illustrative example:

```
date_values = ["2012-01", "2012-02", "2012-04"]
result = validate_monthly_date_range(date_values, start="2012-01", end="2012-04")
```

Expected output shape:

```
{
  "is_complete": False,
  "start": "2012-01",
  "end": "2012-04",
  "n_periods": 3,
  "missing_months": ["2012-03"],
  "extra_months": [],
}
```

Here, `n_periods` is 3 because the input contains three distinct parsed months: 2012-01, 2012-02, and 2012-04.

Dataset connection:

This function is directly useful in Part II when verifying that each required market file contains a complete monthly series from 2012-01 to 2018-12.

### Question 3: 'make\_tidy\_arrival\_table(market\_frames)' [5 marks]

Your task:

Write a function that combines arrival columns from multiple market DataFrames into one tidy table with one row per market-month.

Required function signature:

```
def make_tidy_arrival_table(market_frames):
    pass
```

Input:

- `market_frames`: a dictionary where keys are market names and values are DataFrames containing at least a date column and an arrival column.

Output:

- Return a pandas DataFrame with at least date, market, and arrival columns.

Constraints:

- Preserve all markets provided in `market_frames`.
- Add the market name from the dictionary key.
- Keep only the required tidy columns unless you clearly document additional columns.
- Sort by market and date, or otherwise produce a clearly reproducible order.
- The function may preserve the original date values. Date parsing and monthly-range validation are assessed in Part II. If you parse dates inside this function, do so consistently.

Illustrative example:

```
market_frames = {
  "Australia": pd.DataFrame({"date": ["2012-01", "2012-02"], "arrival": [100, 120]}),
  "Thailand": pd.DataFrame({"date": ["2012-01", "2012-02"], "arrival": [80, 90]}),
}
```

Expected output shape:

| date    | market    | arrival |
|---------|-----------|---------|
| 2012-01 | Australia | 100     |
| 2012-02 | Australia | 120     |
| 2012-01 | Thailand  | 80      |
| 2012-02 | Thailand  | 90      |

Dataset workflow connection:

In Part II, after you load and clean the three required market files, you will have one cleaned DataFrame for each market. Question 3 asks you to write a reusable helper that takes a dictionary of these market-level DataFrames and stacks their arrival series into one tidy table. This tidy table is the same style of object required later as `tidy_arrivals`.

For the real A1 dataset, `market_frames` may correspond to cleaned DataFrames for Australia, United States, and Thailand. Each DataFrame should contain a date column and an arrival column after your cleaning step. The function should add a market column using the dictionary key and return one row per market-month.

The illustrative example uses small toy DataFrames. In Part II, the same idea should scale to the required market files.

#### Question 4: 'summarise\_numeric\_columns(df, exclude=("date",))' [5 marks]

Your task:

Write a function that inspects the columns of a DataFrame and summarises which columns can be treated as numeric analysis columns.

Required function signature:

```
def summarise_numeric_columns(df, exclude=("date", )):
    pass
```

Input:

- `df`: a pandas DataFrame.
- `exclude`: column names to exclude from numeric-column checking.

Output:

- Return a summary DataFrame or dictionary with clear fields.
- Recommended return format: a pandas DataFrame with column, `can_convert_numeric`, `n_missing`, `n_unique`, and `example_value`.

Output field meanings:

- `can_convert_numeric`: True if the column is already numeric or can be converted to numeric without creating invalid values for the non-missing entries.
- `example_value`: one representative non-missing value from the original column, where available.

Constraints:

- Exclude columns listed in `exclude`.
- Determine which remaining columns are numeric or can be converted to numeric.
- `can_convert_numeric` should indicate whether the column is already numeric or can be safely converted to numeric for analysis.
- Include enough information to support schema audit and SII feature identification.

Illustrative example:

```
df = pd.DataFrame({
    "date": ["2012-01", "2012-02"],
    "arrival": [100, 120],
    "hotel search": [0.3, 0.4],
    "note": ["ok", "ok"],
})
```

Expected output shape:

| column       | can_convert | n_missing | n_unique | example_value |
|--------------|-------------|-----------|----------|---------------|
| arrival      | True        | 0         | 2        | 100           |
| hotel search | True        | 0         | 2        | 0.3           |
| note         | False       | 0         | 1        | ok            |

Dataset connection:

This function helps identify arrival and Search Intensity Index columns before feature engineering in Part II.

### Question 5: 'rolling\_mean\_manual(values, window)' [5 marks]

Your task:

Write a function that manually computes a trailing rolling mean for a numeric sequence, returning None until enough previous values are available.

Required function signature:

```
def rolling_mean_manual(values, window):
    pass
```

Input:

- values: a sequence of numeric values.
- window: the trailing window size.

Output:

- Return a list with the same length as values.

Output alignment:

- A trailing window uses the current value and the previous window - 1 values.
- The output index aligns with the input index.
- The first window - 1 positions should be None.

Constraints:

- Before a full window is available, return None.
- If window <= 0, raise ValueError.
- Do not use pandas.rolling() or NumPy rolling helpers.

Illustrative example:

```
values = [10, 20, 30, 40]
window = 3
```

Expected output:



```
[None, None, 20.0, 30.0]
```

Explanation:

The first rolling mean is available at index 2 because values 10, 20, and 30 form the first complete window.

Dataset connection:

This is directly related to the `arrival_roll3_mean` and `arrival_roll12_mean` features in Part II.

### Question 6: 'detect\_local\_extrema(values)' [5 marks]

Your task:

Write a function that finds strict local peaks and valleys in a numeric sequence using neighbouring values.

Required function signature:

```
def detect_local_extrema(values):  
    pass
```

Input:

- values: a sequence of numeric values.

Output:

- Return a dictionary in this form:

```
{"peaks": [indices], "valleys": [indices]}
```

Returned indices:

- Returned indices refer to positions in the original input sequence.
- Strict local extrema use immediate neighbours only.
- Equal neighbouring values do not count.

Constraints:

- Endpoints are not extrema.
- A strict peak occurs when  $\text{values}[i-1] < \text{values}[i] > \text{values}[i+1]$ .
- A strict valley occurs when  $\text{values}[i-1] > \text{values}[i] < \text{values}[i+1]$ .
- Flat neighbours and plateaus are not counted.
- Use zero-based indices.

Illustrative example:

```
values = [3, 5, 4, 4, 6, 2, 3]
```

Expected output:

```
{"peaks": [1, 4], "valleys": [5]}
```

Explanation:

Index 1 is a peak because  $3 < 5 > 4$ . Index 4 is a peak because  $4 < 6 > 2$ . Index 5 is a valley because  $6 > 2 < 3$ . The flat pair at indices 2 and 3 is not counted as a local extremum.

Dataset connection:

This is related to identifying local peaks and valleys in monthly arrivals in Part II.

## Part II: HK2012-2018 Dataset Analysis and Critical Interpretation

Part II is worth 60 marks. Complete Questions 7 to 11 using the three required market files.

| Question    | Component                                                          | Marks |
|-------------|--------------------------------------------------------------------|-------|
| Question 7  | Data access and schema audit                                       | 10    |
| Question 8  | Cleaning and reproducible analysis tables                          | 10    |
| Question 9  | Exploratory data analysis and cross-market comparison              | 15    |
| Question 10 | Feature engineering and SII analysis                               | 15    |
| Question 11 | Critical interpretation, limitations, and project-planning insight | 10    |
|             | Total                                                              | 60    |

Communication, reproducibility, notebook clarity, figure/table labelling, dataset acknowledgement, and avoidance of local absolute paths are assessed within the relevant Part II questions and the final submitted notebook/PDF. They are not a separate scored question.

Part I functions are assessed independently. In Part II, you are encouraged to reuse them where appropriate, but equivalent correct and reproducible implementations are acceptable.

### EDA and visualisation expectations

Exploratory data analysis (EDA) in this assignment means using summary tables, simple derived measures, and clearly labelled visualisations to understand and compare the three required markets before making critical interpretations.

At minimum, your Part II analysis should include:

- one time-series comparison figure showing monthly arrivals for the three required markets;
- one yearly or annual-summary table, such as `annual_summary`;
- one comparison of growth or first-to-last change;
- one comparison of volatility, seasonality, or month-level patterns;
- clearly labelled figure titles, axis labels, legends, and table captions or surrounding explanations.

Visualisations should support your analysis rather than act as decoration. A figure should be referenced in the written discussion and should help explain a specific comparison, trend, pattern, or limitation.

Suitable visualisations may include line charts for monthly trends, bar charts for annual or seasonal comparisons, and scatter plots or small multiples if used carefully for SII-arrival relationships. You are not required to create complex dashboards or interactive visualisations.

## Required analysis objects

Required object names help organise your notebook and help markers locate outputs. Merely creating the object name does not guarantee marks: the object must contain the required structure and evidence for the relevant question. The placeholder value None must be replaced with your completed object.

The exact numerical values are not prescribed in the specification, but the object structure must be clear and reproducible.

The required object structures define minimum assessable evidence. Higher marks require correct computation, clear explanation, appropriate interpretation, and reproducible presentation.

| Object                | Type             | Required structure                                                                                                                                                                                                                                                                                                                                                          | Created in  |
|-----------------------|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| raw_market_data       | dictionary       | Required keys: "Australia", "United States", "Thailand". Values: original DataFrames loaded from the required CSV files. DataFrame index is not prescribed. Each DataFrame must contain identifiable date and arrival columns before cleaning.                                                                                                                              | Question 7  |
| schema_audit          | pandas DataFrame | Required columns: market, rows, columns, date_min, date_max, date_ordered, missing_cells, arrival_numeric, n_numeric_columns, column_issues. Include one row for each required market.                                                                                                                                                                                      | Question 7  |
| clean_market_data     | dictionary       | Same required keys as raw_market_data. Values: cleaned DataFrames with cleaned column labels; dates parsed or clearly parseable; numeric columns converted where appropriate. The cleaned DataFrames should retain the arrival column and original SII-related numeric columns needed for later analysis. DataFrame index is not prescribed.                                | Question 8  |
| tidy_arrivals         | pandas DataFrame | Required columns: date, market, arrival. date should preferably be pandas datetime/month-start timestamp; consistently formatted and parseable date strings are acceptable. Required markets: Australia, United States, Thailand. Expected row count: 252, based on 3 markets x 84 monthly rows, unless a clearly documented data issue is found. One row per market-month. | Question 8  |
| annual_summary        | pandas DataFrame | Required columns: market, year. year should be an integer year such as 2012. Include at least one annual arrival summary column such as annual_total_arrival or annual_mean_arrival. Recommended additional columns: annual_sd_arrival, first_month_arrival, last_month_arrival.                                                                                            | Question 9  |
| feature_table         | pandas DataFrame | Required columns: date, market, arrival, year, month, arrival_diff, arrival_pct_change, arrival_roll3_mean, arrival_roll12_mean, arrival_volatility_3, is_local_peak, is_local_valley.                                                                                                                                                                                      | Question 10 |
| selected_sii_features | list of strings  | At least three selected original SII feature names from the cleaned original CSV columns. The names in selected_sii_features must match columns in the cleaned original data. Do not include date, market, arrival, or derived arrival features.                                                                                                                            | Question 10 |
| sii_summary           | pandas DataFrame | Required columns: market, feature, feature_mean, feature_sd, relationship_measure, relationship_value, interpretation_note. Include one or more rows for each selected SII feature and market. interpretation_note should briefly state what the relationship measure suggests and what it does not prove.                                                                  | Question 10 |

The 252-row expectation for tidy\_arrivals is a structural expectation based on three required markets with 84 monthly rows each.

## Required derived-field definitions

Rolling features and local extrema must be computed within each market, not across the concatenated dataset.

- year: calendar year extracted from date.
- month: calendar month extracted from date, preferably as an integer from 1 to 12.
- arrival\_diff: month-to-month difference in arrival within each market.
- arrival\_pct\_change: month-to-month percentage change in arrival within each market. The first month for each market may be NaN or None.
- arrival\_roll3\_mean: trailing 3-month rolling mean of arrival within each market.
- arrival\_roll12\_mean: trailing 12-month rolling mean of arrival within each market.
- arrival\_volatility\_3: trailing 3-month rolling standard deviation of arrival within each market.
- is\_local\_peak: boolean indicator showing whether the month is a strict local peak in arrival within the same market.
- is\_local\_valley: boolean indicator showing whether the month is a strict local valley in arrival within the same market.

Note on output-shape examples: The tables below show the required output structure only. They are not complete results and they are not fixed values. Your submitted tables must include all required rows for the three required markets. Replace ... with values computed from the dataset.

## Question 7: Data access and schema audit [10 marks]

Your task:

Load the three required market files from the approved raw URL base and audit their structure.

Your notebook should include both code and a short written explanation of what you found during the schema audit.

Required outputs:

- a short explanation of how the files were loaded;
- a schema\_audit table for Australia, the United States, and Thailand;
- row and column counts for each file;
- date range for each file;
- date ordering check;
- missing-cell counts;
- numeric parsing status;
- important column-name issues that may affect analysis;
- clear dataset-source acknowledgement;
- loading code that uses the approved raw URL base and does not rely on local absolute paths.

Example schema-audit table structure, not complete results:

| market        | rows | columns | date_min | date_max | date_order | missing_cells | arrival_num | n_numeric | column_issues |
|---------------|------|---------|----------|----------|------------|---------------|-------------|-----------|---------------|
| Australia     | ...  | ...     | ...      | ...      | ...        | ...           | ...         | ...       | ...           |
| United States | ...  | ...     | ...      | ...      | ...        | ...           | ...         | ...       | ...           |
| Thailand      | ...  | ...     | ...      | ...      | ...        | ...           | ...         | ...       | ...           |

Suggested schema\_audit value meanings:

- market: market name, such as Australia.

- rows: number of rows in the loaded file.
- columns: number of columns in the loaded file.
- date\_min: earliest parsed or parseable date/month.
- date\_max: latest parsed or parseable date/month.
- date\_ordered: boolean value indicating whether dates are already in chronological order.
- missing\_cells: total number of missing cells in the file, or a clearly documented equivalent missingness summary.
- arrival\_numeric: boolean value indicating whether the arrival column is numeric or safely convertible to numeric.
- n\_numeric\_columns: number of columns that are numeric or safely convertible to numeric after excluding date-like columns.
- column\_issues: short text note, such as none, spaces in labels, duplicate-looking labels, or another concise description.

Required object(s):

- raw\_market\_data
- schema\_audit

## Question 8: Cleaning and reproducible analysis tables [10 marks]

Your task:

Clean the loaded data and create reproducible analysis tables for the required markets.

Your notebook should explain the key cleaning decisions you made, especially any column-name changes, date parsing decisions, and numeric conversion decisions.

You may reuse your Part I functions where appropriate. In particular, Question 1 can support column-label normalisation, Question 2 can support monthly date-range validation, and Question 3 can support construction of the tidy arrival table. Equivalent correct implementations are also acceptable.

Required outputs:

- cleaned market-level data that retain the arrival column and original SII-related numeric columns needed for later feature engineering;
- documented original-to-cleaned label mapping;
- parsed and sorted dates;
- verification of complete monthly coverage from 2012-01 to 2018-12;
- numeric conversion where appropriate;
- tidy arrival table with at least date, market, and arrival;
- reproducible cleaning code that can be followed from the submitted notebook.

Example tidy\_arrivals structure, not complete results. Your submitted table should include all required market-month rows:

| date       | market    | arrival |
|------------|-----------|---------|
| 2012-01-01 | Australia | ...     |
| 2012-02-01 | Australia | ...     |
| ...        | ...       | ...     |

Required object(s):

- clean\_market\_data
- tidy\_arrivals

## Question 9: Exploratory data analysis and cross-market comparison [15 marks]

Your task:

Compare the three required markets using exploratory data analysis.

Your notebook should include a short written comparison, supported by your tables and figures, explaining how the three markets differ in volume, growth, volatility, and seasonality.

Use the EDA and visualisation expectations above when preparing your figures, tables, and written comparison.

Required outputs:

- at least one time-series comparison figure;
- clearly labelled figures and tables;
- yearly arrival summary;
- first-to-last change comparison;
- volatility comparison;
- seasonal or month-level summary;
- short evidence-based comparison of similarities and differences in volume, growth, volatility, seasonality, and search-intensity patterns.

As a guide, write 1-2 short paragraphs interpreting the comparison, supported by your tables and figures.

Written answers should refer to specific tables, figures, or computed features from your notebook. Generic discussion that is not grounded in your generated evidence will receive limited credit.

Example annual\_summary structure, not complete results. Your submitted table should cover relevant years for all required markets:

| market        | year | annual_total_arrival | annual_mean_arrival | annual_sd_arrival |
|---------------|------|----------------------|---------------------|-------------------|
| Australia     | 2012 | ...                  | ...                 | ...               |
| United States | 2012 | ...                  | ...                 | ...               |
| Thailand      | 2012 | ...                  | ...                 | ...               |
| ...           | ...  | ...                  | ...                 | ...               |

Required object(s):

- annual\_summary

## Question 10: Feature engineering and SII analysis [15 marks]

Your task:

Create interpretable arrival features and analyse Search Intensity Index signals.

Your notebook should include a short written explanation of why the selected SII features were chosen and what their relationships with arrivals do and do not imply.

As a guide, write 1-2 short paragraphs explaining why the SII features were selected and what the observed relationships do and do not imply.

Search Intensity Index features should be identified from the original cleaned CSV columns before adding your own arrival-derived features. Exclude date, market, arrival, and any derived columns created by you.

The selected SII feature names must be traceable to columns in your cleaned data. Do not submit placeholder names such as <cleaned\_sii\_column\_1> or sii\_feature\_name\_1; your selected feature names must match columns in your cleaned original data.

Students may use correlation or co-movement to help select SII features, but high correlation or co-movement may reflect shared trends, seasonality, scale effects, or confounding. Do not present correlation as causal evidence.

If you use a figure to compare SII-arrival relationships, it should be clearly labelled and interpreted. A table-only comparison is acceptable if it clearly supports the written analysis.

You may reuse your Part I functions where appropriate. In particular, Question 4 can support numeric-column inspection and SII feature identification, Question 5 can support rolling-mean features, and Question 6 can support local peak/valley indicators. Equivalent correct implementations are also acceptable.

Written answers should refer to specific tables, figures, or computed features from your notebook. Generic discussion that is not grounded in your generated evidence will receive limited credit.

Required outputs:

- feature table containing the required derived fields;
- selected\_sii\_features as a list containing at least three selected original SII feature names;
- a transparent feature-selection rule;
- sii\_summary with, for each selected feature and market, interpretable feature summaries and one relationship or co-movement measure with arrivals, such as correlation or another clearly explained measure;
- at least one comparison of an SII-arrival relationship across markets;
- explanation that correlation or co-movement is not causal evidence;
- clearly labelled feature and SII summary tables or figures where used.

Required object(s):

- feature\_table
- selected\_sii\_features
- sii\_summary

Required derived fields:

- year;
- month;
- arrival\_diff;
- arrival\_pct\_change;
- arrival\_roll3\_mean;
- arrival\_roll12\_mean;
- arrival\_volatility\_3;
- is\_local\_peak;
- is\_local\_valley.

Use the derived-field definitions above. In particular, rolling features, volatility, and local peak/valley indicators must be computed within each market.

Example feature\_table structure, not complete results. Values shown as ... must be computed from the dataset:

| date      | market    | arrival | year | month | arrival_di | arrival_pc | arrival_ro | arrival_ro | arrival_vc | is_local_p | is_local_valley |
|-----------|-----------|---------|------|-------|------------|------------|------------|------------|------------|------------|-----------------|
| 2012-01-0 | Australia | ...     | 2012 | 1     | ...        | ...        | ...        | ...        | ...        | False      | False           |
| 2012-02-0 | Australia | ...     | 2012 | 2     | ...        | ...        | ...        | ...        | ...        | False      | False           |
| ...       | ...       | ...     | ...  | ...   | ...        | ...        | ...        | ...        | ...        | ...        | ...             |

Example selected\_sii\_features structure. The feature names shown are structure-only placeholders, not actual required SII feature names:

```
# Example structure only. Replace these with real column names from your cleaned data.
selected_sii_features = ["<cleaned_sii_column_1>", "<cleaned_sii_column_2>", "<cleaned_sii_column_3>"]
```

Do not submit placeholder names such as <cleaned\_sii\_column\_1> or sii\_feature\_name\_1; your selected feature names must match columns in your cleaned original data.

Example sii\_summary structure, not complete results. The feature names shown are structure-only placeholders, not actual required SII feature names:

| market        | feature            | feature_mean | feature_sd | relationship_mea | relationship_valu | interpretation_note |
|---------------|--------------------|--------------|------------|------------------|-------------------|---------------------|
| Australia     | <cleaned_sii_colun | ...          | ...        | correlation      | ...               | ...                 |
| United States | <cleaned_sii_colun | ...          | ...        | correlation      | ...               | ...                 |
| Thailand      | <cleaned_sii_colun | ...          | ...        | correlation      | ...               | ...                 |
| ...           | ...                | ...          | ...        | ...              | ...               | ...                 |

### Question 11: Critical interpretation, limitations, and project-planning insight [10 marks]

Your task:

Write a short evidence-based analysis in your notebook and exported PDF. This question is primarily assessed through your written interpretation, not through additional code.

As a guide, write approximately 300-500 words, or 3-5 concise paragraphs, supported by your tables and figures.

Written answers should refer to specific tables, figures, or computed features from your notebook. Generic discussion that is not grounded in your generated evidence will receive limited credit.

Required written answer:

- main findings about the three required markets;
- what the Search Intensity Index signals can support;
- what the Search Intensity Index signals cannot support;
- limitations of the data and analysis;
- what a later modelling or forecasting project would need to handle;
- clear links to the relevant tables and figures in your notebook and exported PDF.

Formal forecasting is outside Assignment 1. Do not replace the required descriptive analysis with a forecasting model.

## Part III: Transfer and Reflection

Part III is worth 10 marks. Complete Question 12 as a discussion/reflection task.

You are not required to download, clean, transform, model, or fully analyse the ISF-TDF2023 dataset for Assignment 1.

Reference dataset:

<https://github.com/tulip-lab/open-data/blob/main/ISF-TDF2023/README.md>

At the time of preparing this assignment, the ISF-TDF2023 README describes a monthly Chinese outbound tourism-demand dataset with multiple destination series and documented historical, validation, and evaluation periods. Base your discussion on the README available when you complete the assignment.



## Question 12: Transfer and reflection [10 marks]

Your task:

Write a short reflection explaining how the functions, objects, and workflow you developed in Part I and Part II could be transferred or adapted to the ISF-TDF2023 dataset.

This is a reflection task only. Do not add a full ISF-TDF2023 data-loading, cleaning, analysis, modelling, or forecasting workflow to your notebook.

As a guide, write approximately 250-400 words, or 2-4 concise paragraphs.

Required written answer:

- identify which Part I functions could transfer directly and which would need adaptation;
- explain which Part II objects or workflow stages have a useful equivalent for a multi-destination monthly tourism-demand dataset;
- discuss at least two dataset differences that would affect transfer, such as the number of destination series, different date coverage, historical/validation/evaluation periods, missing values, or forecasting-evaluation context;
- explain how you would check the new dataset before reusing your A1 workflow;
- state one limitation or risk of transferring a workflow from HK2012-2018 to ISF-TDF2023 without redesign.

Your answer should refer to the public ISF-TDF2023 README at a high level. It should not include computed results from the ISF-TDF2023 CSV file.

## Back matter

---

### Submission requirements

Submit the following files in CloudDeakin via Assignments located under Assessment:

- SIT742-2026T2-A1-<StudentID>.ipynb
- SIT742-2026T2-A1-<StudentID>.pdf

The PDF must be exported from your completed notebook and must include code outputs, figures, tables, and written explanations. You are not required to prepare a separate full report.

Students aiming for High Distinction are encouraged to submit a short video as supporting evidence. Submit the video together with the notebook and exported PDF if you choose to include it or if the teaching team requests it for verification.

For general help with CloudDeakin assignment submission, refer to:

<https://www.deakin.edu.au/students/help/about-clouddeakin/assessment/assignments>

### File naming convention

Replace <StudentID> with your Deakin student ID.

```
SIT742-2026T2-A1-<StudentID>.ipynb
SIT742-2026T2-A1-<StudentID>.pdf
```

If a video is submitted or requested, use:

```
SIT742-2026T2-A1-<StudentID>-video.<ext>
```

## Video evidence

Students aiming for High Distinction are encouraged to submit a short video as supporting evidence. The video can help demonstrate your code understanding, reproducibility, and ability to explain the main analytical decisions in your own words.

The video is not a separate scored component and is not a substitute for the completed notebook and exported PDF. Not submitting a video does not automatically reduce your mark. However, if the teaching team requests video evidence for authorship, reproducibility, or code-understanding verification and the requested evidence is not provided, this may affect the relevant verification process.

Recommended video requirements:

- length: 3-5 minutes;
- file size: 20 MB or less where possible;
- compression: use H.265/HEVC or another efficient compression format where possible;
- format: screen recording with voice and face camera visible;
- submit the video file together with the notebook and exported PDF;
- filename: SIT742-2026T2-A1-<StudentID>-video.<ext>.

If you have a technical, accessibility, or approved privacy-related reason that prevents this format, follow the unit-site instructions or contact the teaching team before submission.

Your video should briefly cover:

- the structure of your notebook;
- evidence that your notebook can run;
- explanation of one or two Part I functions;
- explanation of the main Part II data workflow;
- one or two key findings;
- one limitation of your analysis;
- if relevant, how GenAI was used and how you checked, revised, and understood the final work.

## Reproducibility requirements

Before submission, confirm that:

- the notebook runs from top to bottom in a clean runtime;
- the approved raw URL base is used;
- data-loading code does not depend on local absolute paths;
- all required output objects are created;
- figures and tables are generated by the submitted notebook;
- the Part III reflection is included;
- package imports are clear and necessary;
- the dataset source is acknowledged;
- the exported PDF reflects the completed notebook.

## Academic integrity and GenAI policy

You must complete this assignment in accordance with Deakin's academic integrity requirements and the current SIT742 unit-site instructions.

If you use generative AI tools in preparing your work, you are responsible for checking, revising, understanding, and acknowledging that use according to the unit-site instructions and Deakin guidance. Generative AI output must not replace your own understanding, analysis, or explanation.

For current guidance, refer to:

- Deakin academic integrity: <https://www.deakin.edu.au/students/study-support/academic-integrity>

- Using generative AI in your studies:  
<https://www.deakin.edu.au/students/study-support/study-resources/artificial-intelligence>
- The SIT742 unit site on CloudDeakin

## **Extensions, special consideration, and submission policy**

For extensions, special consideration, late submission rules, and CloudDeakin submission support, follow the current SIT742 unit-site instructions and official Deakin student-support pages.

For current guidance, refer to:

- Assessments, extensions, due dates, and late penalties:  
<https://www.deakin.edu.au/students/study-support/assessments-and-examinations/assessments>
- CloudDeakin assignment submission help:  
<https://www.deakin.edu.au/students/help/about-clouddeakin/assessment/assignments>
- The SIT742 unit site on CloudDeakin

## **Submission checklist**

Before submitting, check that:

- your completed notebook is included;
- your exported PDF is included;
- filenames follow the required pattern;
- files are submitted in CloudDeakin via Assessment > Assignments;
- the notebook runs from top to bottom;
- the PDF contains code outputs, figures, tables, and explanations;
- all required output objects are present;
- the Part III reflection is included;
- the three required market files are used;
- optional extension markets, if used, are clearly identified as optional;
- no local absolute paths, credentials, or private data are included;
- the dataset source is cited;
- any video evidence follows the requirements above and any later unit-site instructions.